P0401R2
Providing size feedback in the Allocator interface

Published Proposal, 2020-01-12

This version:
http://wg21.link/P0401R2

Authors:
Jonathan Wakely
Chris Kennelly (Google)

Audience:
LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Utilize size feedback from Allocator to reduce spurious reallocations

Table of Contents

§ 1. Introduction

This is a library paper to describe how size feedback can be used with allocators:

- In the case of `std::allocator`, the language feature proposed in [P0901R5] could be used.
- In the case of custom allocators, the availability of the traits interface allows standard library containers to leverage this functionality.

§ 2. Motivation

Consider code adding elements to `vector`:

```
std::vector<int> v = {1, 2, 3};
// Expected: v.capacity() == 3

// Add an additional element, triggering a reallocation.
v.push_back(4);
```

Many allocators only allocate in fixed-sized chunks of memory, rounding up requests. Our underlying heap allocator received a request for 12 bytes (`3 * sizeof(int)`) while constructing `v`. For several implementations, this request is turned into a 16 byte region.

§ 2.1. Why not `realloc`

`realloc` poses several problems problems:

- We have unclear strategies for growing a container. Consider the two options (`a` and `b`) outlined in the comments of the example below. Assume, as with the §2 Motivation example, that there is a 16 byte size class for our underlying allocator.

  ```
  std::vector<int> v = {1, 2, 3};
  // Expected: v.capacity() == 3

  // Add an additional element.  Does the vector:
  // a) realloc(sizeof(int) * 4), adding a single element
  // b) realloc(sizeof(int) * 6), attempt to double, needing reallocation
  v.push_back(4);

  // Add another element.
  // a) realloc(sizeof(int) * 5), fails in-place, requires reallocation
  // b) size + 1 <= capacity, no reallocation required
  v.push_back(5);
  ```

  Option `a` requires every append (after the initial allocation) try to `realloc`, requiring an interaction with the allocator each time.

  Option `b` requires guesswork to probe the allocator's true size. By doubling (the typical growth pattern for `std::vector` in libc++ and libstdc++), we skip past the existing allocation's size boundary. While we can use a reduced growth rate, doing so requires more interactions with the allocator (calls to `realloc`), as well as copies (when we cannot resize in-place).

  Both of these options add extra branching on the control flow for growing the buffer.

- When the allocator cannot resize the buffer in-place, it allocates new memory and `memcpy`'s the contents. For non-trivially copyable types, this is disastrous. The *language* lacks support for reallocating in-place *without* moving. While this function could be implemented as magic library function, it could not interact with many underlying `malloc` implementations (libc, in the case of libc++ and libstdc++) used to build `operator new`, nor could it interact with user-provided replacement functions.

  In-place reallocation has been considered in [N3495] (throwing `std::bad_alloc` when unable) and [P0894R1] (returning `false` when unable). This has sometimes been referred to as an "extend" or an "expand" interface. Extending an existing allocation requires a round-trip to the allocator (and its requisite branch/call overheads) to determine whether the allocation *could* be expanded. As discussed previously, we need to probe for the true size of the backing object (until we find extend fails) until we give up and apply a multiplicative expansion factor.

For fixed-size allocators it makes more sense, and is much simpler for containers to adapt to, if the allocator is able to over-allocate on the initial request and inform the caller how much memory was made available.

2.2. Why not ask the allocator for the size

We could also explore APIs that answer the question: "If I ask for `N` bytes, how many do I actually get?" [jemalloc] calls this `nallocx`.

- `nallocx` requires an extra call / size calculation. This requires we duplicate work, as the size calculation is performed as part of allocation itself.
- `nallocx` impairs telemetry for monitoring allocation request sizes. If a `nallocx` return value is cached, the user appears to be asking for exactly as many bytes as `nallocx` said they would get.

See also [P0901R5]'s discussion "`nallocx`: not as awesome as it looks."

3. Proposal

Wording relative to [N4800].

We propose a free function to return the allocation size simultaneously with an allocation.

> Amend [allocator.requirements]:
>
> ```
> template<typename T = void>
> struct sized_ptr_t {
>   T *ptr;
>   size_t n;
> };
>
> template<typename T, typename Allocator = std::allocator<T>>
> sized_ptr_t<T> allocate_at_least(Allocator& a, size_t n);
> ```
>
> - *Effects:* Let `s` be the result of `allocate_at_least(a, n)`. Memory is allocated for at least `n` objects of type `T` but objects are not constructed.
> - *Returns:* `sized_ptr_t{ptr, m}`, where `ptr` is that memory and `m` is the number of objects for which memory has been allocated, such that `m>=n`.
> - [*Note:* If `s.n == 0`, the value of `s.ptr` is unspecified. —end note]
>
> Table 34 - Cpp17Allocator Requirements
>
> | | | |
> |---|---|---|
> | `a.deallocate(p,n)` | (not used) | *Requires*: `p` shall be a value returned by an earlier call to `allocate` or `allocate_at_least` that has not been invalidated by an intervening call to `deallocate`. If this memory was obtained by a call to `allocate`, `n` shall match the value passed to `allocate` to obtain this memory. Otherwise, `n` shall satisfy `capacity >= n >= requested` where `[p, capacity] = allocate_at_least(requested)` was used to obtain this memory.

Throws: Nothing. |

4. Design Considerations

4.1. `allocate` selection

There are multiple approaches here:

- Return a pointer-size pair, as presented.
- Overload `allocate` and return via a reference parameter. This potentially hampers optimization depending on ABI, as the value is returned via memory rather than a register.

4.2. `deallocate` changes

We now require flexibility on the size we pass to `deallocate`. For container types using this allocator interface, they are faced with the choice of storing *both* the original size request as well as the provided size (requiring additional auxillary storage), or being unable to reproduce one during the call to `deallocate`.

As the true size is expected to be useful for the `capacity` of a `string` or `vector` instance, the returned size is available without additional storage requirements. The original (requested) size is unavailable, so we relax the `deallocate` size parameter requirements for these allocations.

4.3. Interaction with `polymorphic_allocator`

`std::pmr::memory_resource` is implemented using virtual functions. Adding new methods, such as the proposed allocate API would require taking an ABI break.

5. Revision History

5.1. R1 → R2

Applied LEWG feedback from [Cologne].

> Poll: We want a feature like this.
>
SF	F	N	A	SA
2	9	2	0	0

- As users may specialize `std::allocator_traits`, this revision moves to a free function `std::allocate_at_least` as suggested via a LEWG poll.

> Poll: Name choice
>
5	`allocate_with_size`
4	`overallocate`
2	`allocate_with_oversubscribe`
6	`allocate_oversize`
14	`allocate_at_least`

> Poll: Prefer a struct rather than a requirement of structured bindings.
>
SF	F	N	A	SA
2	8	0	2	1

References

Informative References

[Cologne]
Cologne Meeting Minutes. 2019-07-17. URL: http://wiki.edg.com/bin/view/Wg21cologne2019/P0401

[JEMALLOC]
jemalloc(3) - Linux man page. URL: http://jemalloc.net/jemalloc.3.html

[N3495]
Ariane van der Steldt. inplace realloc. 7 December 2012. URL: https://wg21.link/n3495

[N4800]
Working Draft, Standard for Programming Language C++. 2019-01-21. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf

[P0894R1]
realloc for C++. 2019-01-18. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0894r1.md

[P0901R5]
Size feedback in operator new. 2019-10-03. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0901r5.html